Derek Lau

February 22, 2026

Foundations Of Python Programming

Assignment05

GitHubURL: https://github.com/derek-edu/IntroToProg-Python-Mod05

Incorporating Dictionaries and JSON Processing

Introduction

The objective of this assignment was to transition the student registration program toward advanced data management using dictionaries and JSON processing. This updated version enabled structured data storage and formal error handling. This update allowed for more complex data organization and ensured the program could survive unexpected user inputs or file system issues through defensive programming techniques.

Defining Data Constants and Variables

To implement this structured approach, specific constants and variables were defined. The constants ***MENU*** and ***FILE_NAME*** were established to maintain a consistent interface and file reference. The ***students*** variable was created as an empty list to serve as the master collection, while ***student_data*** was redefined as an empty dictionary to store individual records. Additional variables for student names and course details were initialized as empty strings to facilitate data collection during the registration process (Figure 1).

```
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
-----------------------------------------
'''
# Define the Data Constants
FILE_NAME: str = "Enrollments.json"

# Define the Data Variables and constants
student_first_name: str = ''  # Holds the first name of a student entered by the user.
student_last_name: str = ''  # Holds the last name of a student entered by the user.
course_name: str = ''  # Holds the name of a course entered by the user.
student_data: dict = {}  # one row of student data (TODO: Change this to a Dictionary)
students: list = []  # a table of student data
file = None  # Holds a reference to an opened file.
menu_choice: str  # Hold the choice made by the user.
```

Figure 1: Defining constants and variables

Automatic JSON Loading and Structured Error Handling

Upon execution, a critical processing step was performed to automatically read existing data from the ***Enrollments.json*** file. The ***json.load()*** function was utilized to transform the file contents directly into Python dictionaries. To ensure program stability, structured error handling was implemented using ***try-except*** blocks. This logic was designed to catch ***FileNotFoundError*** or ***JSONDecodeError***, allowing the script to initialize an empty list and continue running rather than terminating the entire operation if the source file was missing or invalid (Figure 2).

```
try:

    file = open(FILE_NAME, "r")
    students = json.load(file)

except FileNotFoundError as e:
    print("JSON file must exist before running this script!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
    students: list = [] # Resetting the students list
except json.JSONDecodeError as e:
    print("JSON file exists but the data is invalid!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
    students: list = [] # Resetting the students list
except Exception as e:
    print("There was a non-specific error!\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
    students: list = [] # Resetting the students list
finally:
    # Check if a file object exists and is still open
    if file is not None and file.closed == False:
        file.close()
```

Figure 2: Initial JSON Reading and Error Handling

Processing Registrations with Input Validation

The logic for capturing new registrations was refined to incorporate dictionary assignment and input validation. When Option 1 is selected, the program prompts for student details. To meet the requirement for structured error handling, ***if*** statements and the ***raise ValueError*** command were utilized to verify that names contained only alphabetical characters. Once validated, these values were assigned to the ***student_data*** dictionary and appended to the master ***students*** list. A confirmation message is then displayed to indicate a successful update to the memory (Figure 3).

```python
if menu_choice == "1":  # This will not work if it is an integer!
    try:
        student_first_name = input("Enter the student's first name: ")
        if not student_first_name.isalpha():
            raise ValueError("The first name should not contain numbers.")
        student_last_name = input("Enter the student's last name: ")
        if not student_last_name.isalpha():
            raise ValueError("The last name should not contain numbers.")
        course_name = input("Please enter the name of the course: ")
        student_data = {"FirstName": student_first_name,
                        "LastName": student_last_name,
                        "CourseName": course_name}
        students.append(student_data)
        print(f"\nYou have registered {student_first_name} {student_last_name} for {course_name}.")
    except ValueError as e:
        print(e)  # Prints the custom message
        print("-- Technical Error Message -- ")
        print(e.__doc__)
        print(e.__str__())
    except Exception as e:
        print("There was a non-specific error!\n")
        print("-- Technical Error Message -- ")
        print(e, e.__doc__, type(e), sep='\n')
```

Figure 3: Dictionary Creation and User Input Validation

Permanent Registration in JSON File

Registered student information was saved into a JSON file instead of a CSV file in the previous version. For Option 3, the ***Enrollments.json*** file was opened in write mode, and the ***json.dump()*** function was used to record the entire ***students*** list. This process was wrapped in a ***try-except-finally*** block to handle potential errors during the writing process. To satisfy the specific technical requirements, the ***.close()*** method was explicitly called within the ***finally*** block, ensuring the file resource was released regardless of whether the save operation succeeded or failed. Following the save, a summary of the current records was presented to the user (Figure 4).

```python
try:
    file = open(FILE_NAME, "w")
    json.dump(students, file)
    print("The following data was saved to file!")
    print("-" * 50)
    for student in students:
        print(f"{student['FirstName']},{student['LastName']},{student['CourseName']}")
    print("-" * 50)
except TypeError as e:
    print("Please check that the data is a valid JSON format\n")
    print("-- Technical Error Message -- ")
    print(e, e.__doc__, type(e), sep='\n')
except Exception as e:
    print("-- Technical Error Message -- ")
    print("Built-In Python error info: ")
    print(e, e.__doc__, type(e), sep='\n')
finally:
    if file is not None and file.closed == False:
        file.close()
```

Figure 4: JSON Save Process and Verification

Testing and Execution

The script was tested for compatibility in both the PyCharm editor and the Windows Command Prompt (cmd). Testing focused on verifying the error handling logic by intentionally providing numeric input for names and attempting to load invalid JSON files. In each scenario, the structured error handling correctly caught the exceptions and informed the user without crashing the program. Final verification was performed by inspecting the ***Enrollments.json*** file in a text editor to confirm that the dictionary objects were correctly formatted with proper syntax (Figure 5).

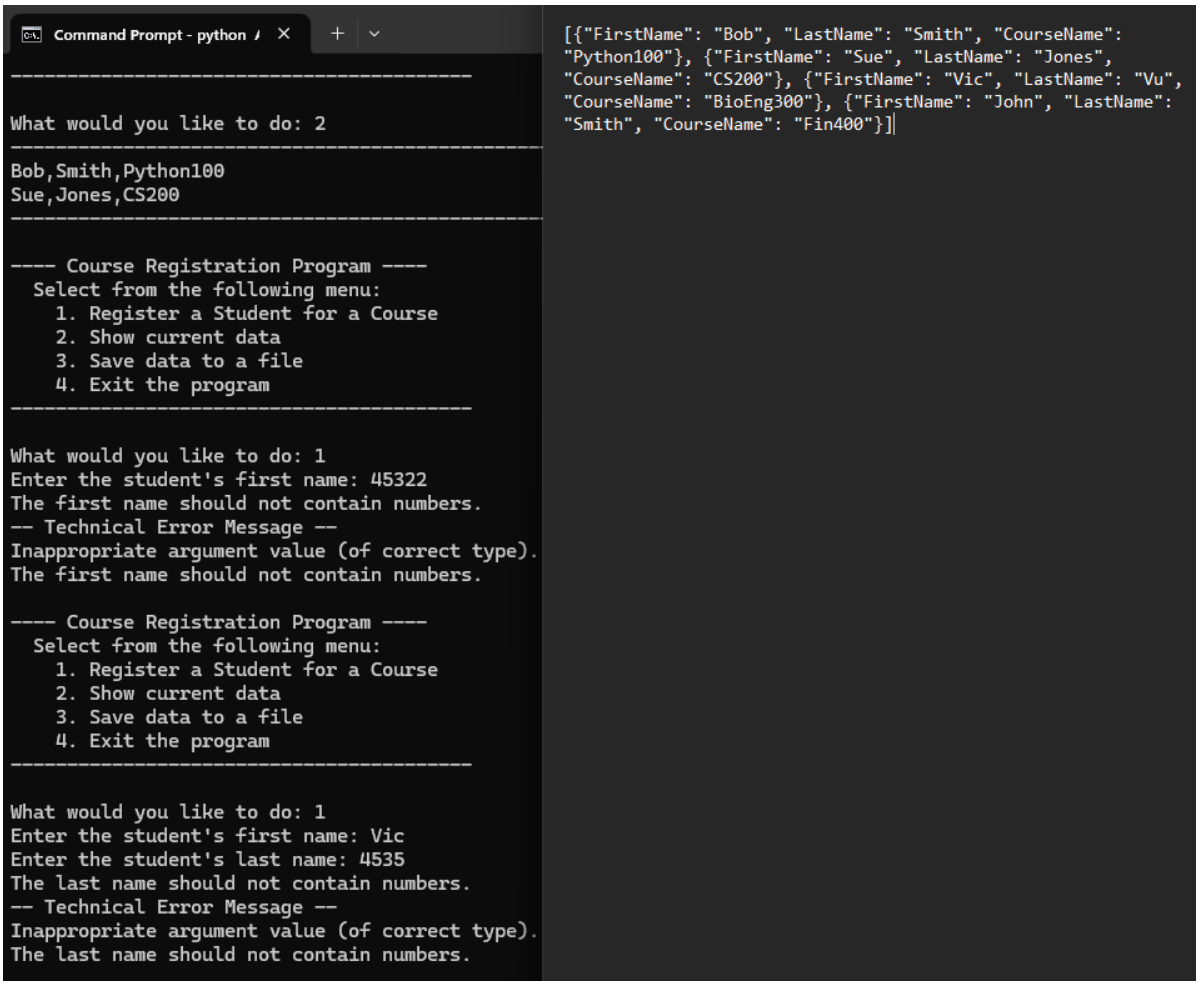

Figure 5: Execution in cmd and JSON Output

Summary

This assignment demonstrated the advantages of using dictionaries and JSON for managing structured data within a Python application. By implementing defensive programming through ***try-except*** blocks and validating user input, the script was made significantly more robust. The project successfully combined automated data validation with permanent file storage, resulting in a more resilient student registration tool.